# Predicting Codeforces tags

**Multi-label classification of 4,982 exercises**

13 methods · one protocol · one command line

Ph.D. Max Sobroza

October 2026

# The task

## One exercise in, any of the 8 focus tags out

### Input: one Codeforces exercise

- Statement, notes and sample tests
- An accepted Python solution
- A quarter of them in Python 2

### Output: any number of 8 tags

math · graphs · strings · number theory · trees · geometry · games · probabilities

46% of exercises have none of the 8.

**4,982**
exercises [1]

**Macro F1**
mean of 8 per-tag F1

**< 10 s**
for one prediction

**1 GPU**
RTX 2080 Ti, 11 GB

**One CLI**
methods, predict, evaluate

[1] Khan et al., *XCodeEval*, ACL 2024 (arXiv:2303.03004). Data: CC BY-NC 4.0.

01

# The data

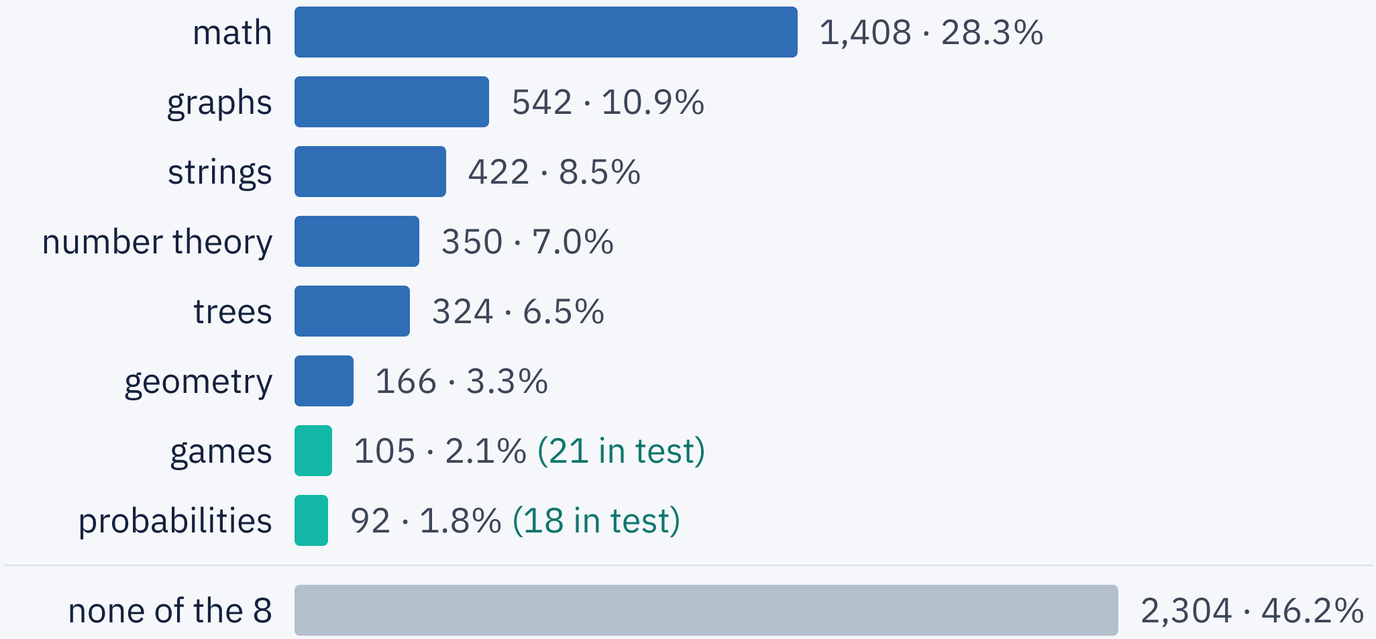
Tag frequency · tags per problem · tags together

DATA ANALYSIS

# How often each tag appears

## Every tag is a minority; 46% carry none

Problems carrying each focus tag, of 4,982




### 15× math vs probabilities

Macro F1 headline, per-tag scores beside it

### Thresholds matter

Logistic regression: 0.19 at 0.5, 0.61 per-tag

### Rare tags, small test sets

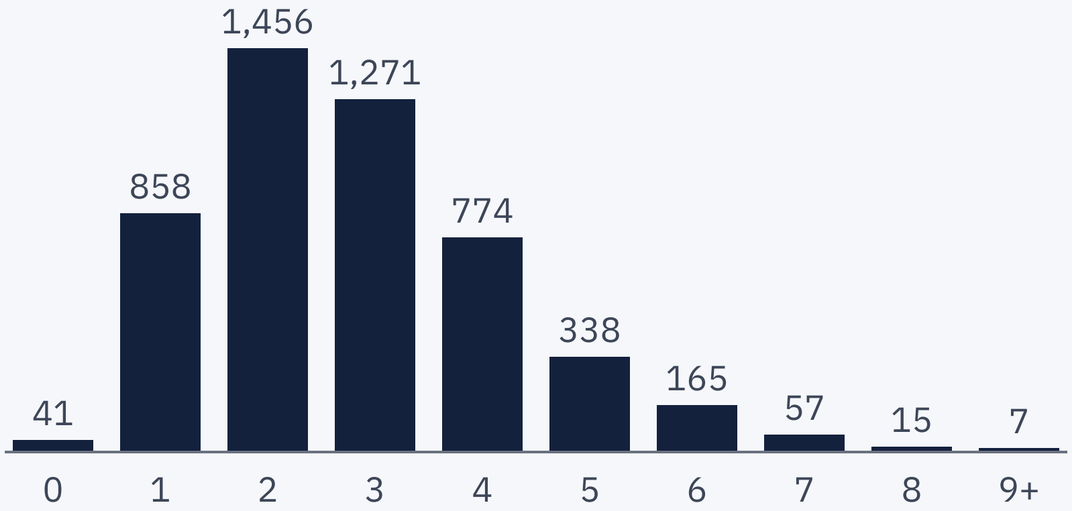
So scores carry bootstrap intervals

DATA ANALYSIS

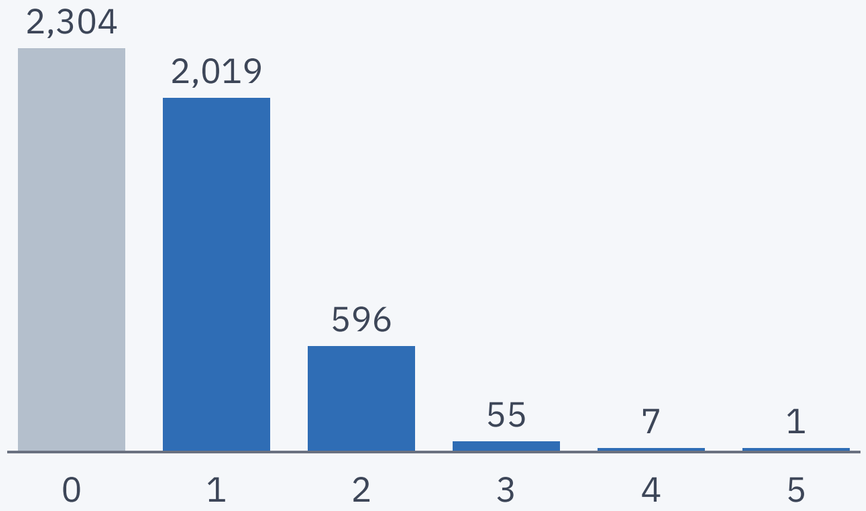
# Tags per problem

## One decision per tag; an empty answer is allowed

**All 37 tags per problem** · mean 2.8, median 3




**The 8 focus tags per problem** · mean 0.68




### Mostly one focus tag

75% of tagged problems carry exactly one of the 8

### Focus tags come with others

89% also carry a non-focus tag, e.g. greedy, dp

DATA ANALYSIS

# Which tags appear together

## Math is the hub; strings stands alone

**Each row:** share of its problems that also carry the column tag

| | math | graphs | strings | number theory | trees | geometry | games | probabilities |
|---|---|---|---|---|---|---|---|---|
| math | **100%** | 5% | 3% | 19% | 3% | 4% | 2% | 3% |
| graphs | 13% | **100%** | 2% | 4% | **23%** | 1% | 1% | 1% |
| strings | 9% | 3% | **100%** | 2% | 4% | 0% | 1% | 0% |
| number theory | **75%** | 7% | 2% | **100%** | 4% | 1% | 2% | 3% |
| trees | 12% | **38%** | 5% | 4% | **100%** | 1% | 3% | 3% |
| geometry | **36%** | 4% | 1% | 3% | 2% | **100%** | 1% | 0% |
| games | **28%** | 6% | 6% | 8% | 9% | 2% | **100%** | 1% |
| probabilities | **49%** | 7% | 0% | 10% | 10% | 0% | 1% | **100%** |

### Not symmetric

Only 19% of math is number theory

### Graphs and trees

38% of trees are also graphs

### So

The split balances the 9 frequent pairs

02

# The protocol

Duplicates · split · pipeline · reproducibility · model choice

DATA QUALITY

# Duplicate problems

## 82 version pairs; a random split leaks about 26

EASY · B1 · DIFFICULTY 800

### Wonderful Coloring 1

Paint a string's letters in two colours; print how many are red.

strings · greedy

TF-IDF cosine

**0.497**

0.8 misses it

LLMs: **versions**

→ **one group**

HARD · B2 · DIFFICULTY 1400

### Wonderful Coloring 2

Same rules on integers with k colours; print the colouring.

greedy · binary search · constructive algorithms · data structures

| Detector | Missed of 82 | Largest group |
|---|---|---|
| TF-IDF cosine ≥ 0.8 | 11 | 2 |
| TF-IDF cosine ≥ 0.4 | 0 | 11, chained |
| **0.4 + two LLM verdicts** | **0** | **2** |

### Labels disagree

strings only on the easy version

# Train and test split

## Test mirrors the data; no duplicate pair is split

**4,982 problems**
one Python solution each

→

**Near-duplicates**
203 TF-IDF pairs, 82 kept by LLMs

→

**4,900 groups**
82 pairs, 4,818 singletons

→

**Stratify [2]**
whole groups; 8 tags, 9 pairs, year

Dev · 3,988 problems: train and tune, in 5 folds of 790 to 806

Test · 994 (20%), once




| Share of problems | All | Test |
|---|---|---|
| math | 28.3% | 28.3% |
| number theory (worst) | 7.0% | 7.2% |
| none of the 8 | 46.2% | 46.2% |

| Against 50 random splits | Frozen | Random |
|---|---|---|
| Duplicate pairs split | **0** | 25.8 |
| Worst tag share off by | **3.1%** | 28.1% |
| Year mix off by | **0.6%** | 3.7% |

[2] K. Sechidis, G. Tsoumakas, I. Vlahavas. *On the Stratification of Multi-Label Data*. ECML PKDD 2011.

# The pipeline

## One pipeline for every method; one prediction in 7 ms

### BUILT ONCE, FOR EVERY METHOD

**Data** → **Frozen split** → **Tune on dev** → **Freeze, test once** → **Verify · MLflow · CLI**

### ONE PREDICTION, WITH TFIDF

**Raw problem**
statement, notes, solution

→

**Preprocess**
same as in training

→

**TF-IDF blocks**
statement: 43,354
code: 2,354, × 0.22

→

**8 linear SVMs**
a score and threshold per tag

→

**Tags**
0 to 8, in 7 ms

**203 Optuna trials**
Sample I/O cues dropped: −0.014 macro F1

**Code never parsed**
Regex tokens read Python 2 and 3 alike

# Reproducibility

```
# 1 · One sklearn Pipeline: raw problems in, a score per tag out
pipeline = Pipeline([
    ("preprocess", FunctionTransformer(raw_to_problems)),
    ("features", feature_blocks(params)),  # statement + code TF-IDF
    ("classifier", OneVsRestClassifier(
        LinearSVC(C=params.c, random_state=MODEL_SEED))),
])

# 2 · Thresholds cross-fitted on the 5 frozen dev folds, then refit
model = CrossFittedTagThresholds(pipeline, refit=True)
model.fit(dev_raw, dev_labels, groups=dev_raw["fold"])
skops.io.dump(model, folder / "model.skops")

# 3 · Fingerprint; log_method logs once per recipe, in its own run
record = make_run_record("tfidf", recipe_sha256, allow_dirty=False,
                         data_dir=data, split_path=split)
mlflow.pyfunc.log_model(                    # models from code
    name="tagging_method",
    python_model="tracking/pyfunc_model.py",
    artifacts={"method": str(folder)},
    signature=raw_input_signature(tags, input_columns),
    pip_requirements=method_requirements("tfidf"),
    params=flat_record(record))
```

### 1 · Same object everywhere

Same code to train, verify and `predict --method tfidf`.

### 2 · Honest and repeatable

Freeze refuses to write unless it reproduces the study's F1.

### 3 · Pinned in MLflow

The run record pins commit, data, split and environment.

# How methods are chosen

## Choose on dev out of fold; score test once

### 1 Out-of-fold scores

Fit on four folds, score the fifth

### 2 Cross-fitted thresholds

Each fold's threshold is chosen on the other folds

### 3 The test set, once

994 test predictions: reported, never used to choose

## How each family learns

| Family | Training and tuning |
|---|---|
| Label prior | the tags seen in training, for every problem |
| TF-IDF | 203 Optuna trials on dev out-of-fold macro F1 |
| Cross-encoder classifier (ModernBERT) | tuned on one dev fold, then a bag of five fold models |
| Dense embeddings (F2LLM) | a grid study of heads over frozen embeddings |
| LLM-based (Code2Tag, DesCode2Tag) | the xCodeEval paper's fixed prompts: nothing learned |
| TypeSafe Jev | fixed questions and definitions: only the thresholds |

# 03

# Results

The paper · score against speed · all methods

XCODEEVAL, KHAN ET AL., ACL 2024

# Comparison with the paper

| Setting | Model | Fine-tuned? | All tags, dev / test | 8 focus tags, test |
|---|---|---|---|---|
| Paper · LLM-based (Code2Tag) | gpt-3.5-turbo-0301 | No: zero-shot | 29.45 (its validation) | not reported |
| Paper · LLM-based (DesCode2Tag) | gpt-3.5-turbo-0301 | No: zero-shot | 33.73 (its validation) | not reported |
| Ours · code2tag-text | gpt-6-luna | No: zero-shot | 46.8 / 47.0 | 63.1 |
| Ours · descode2tag-text | gpt-6-luna | No: zero-shot | 49.8 / 50.4 | 72.1 |
| Ours · descode2tag-json | gpt-6-luna + schema | No: zero-shot | 49.0 / 51.5 | 74.1 |
| Untuned logistic regression | TF-IDF of the statement | Yes | 33.6 / 36.5 | 64.7 |
| **jev-statement-code (CLI default)** | TypeSafe jev-1.13.0 | No: thresholds | 8 tags only | **75.1** |

### Same direction

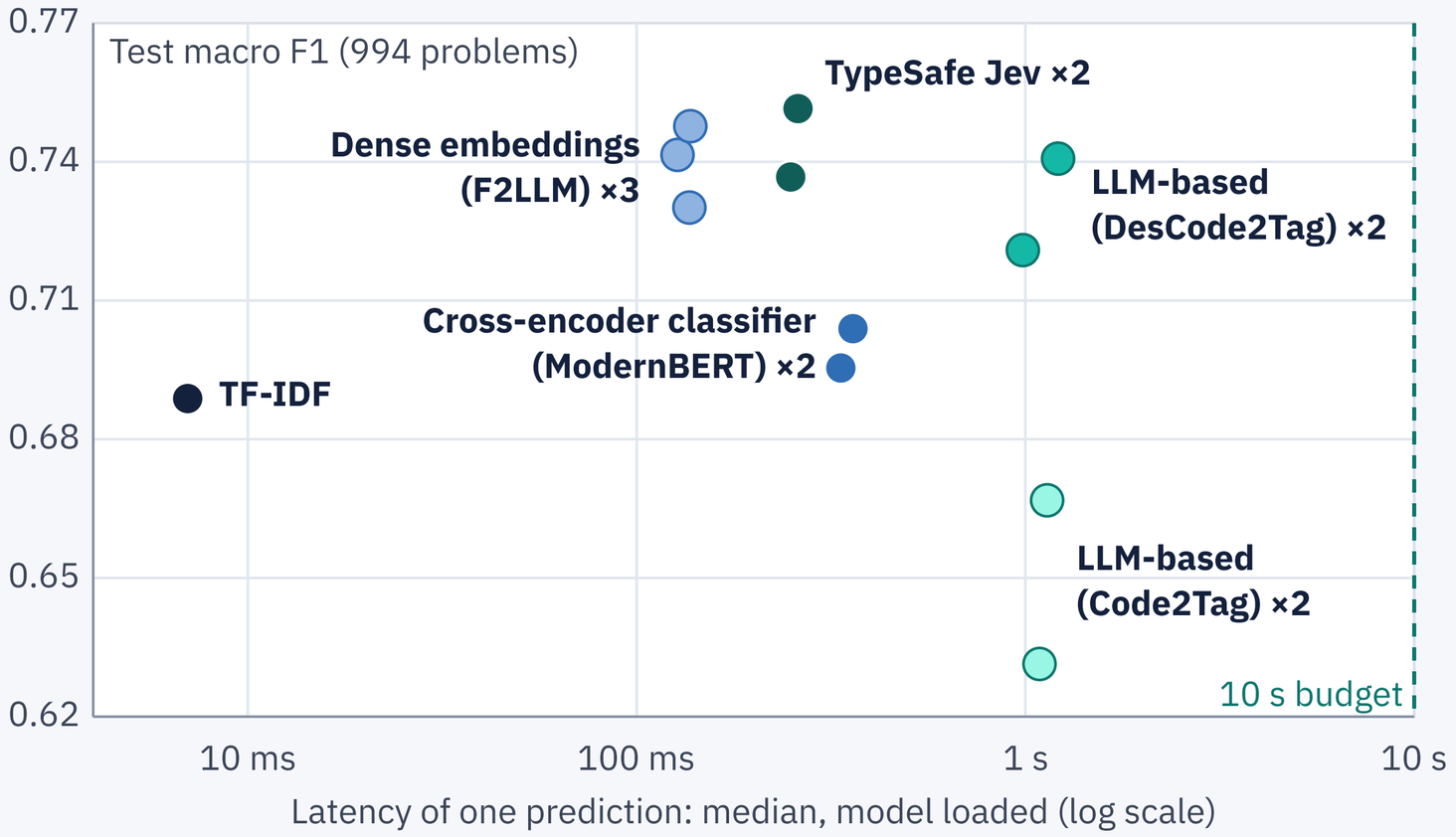
Statement adds 4.3 points in the paper, 3.4 here

### Not like-for-like

31 vs 37 tags, rebuilt prompts, newer model

# Score against speed

## All under the 10 s budget on a local RTX 2080 Ti (11 GB)




### Cost per 1k predictions

| Family | Cost |
|---|---|
| TF-IDF, prior | free, CPU |
| Cross-encoder classifier (ModernBERT) | free, GPU |
| Dense embeddings (F2LLM) | free, GPU |
| LLM-based (Code2Tag) | $0.06–0.08 |
| LLM-based (DesCode2Tag) | $0.11–0.13 |
| TypeSafe Jev | $0.12–0.13 |

# All methods

| Method | How it works | Fine-tuned? | Params | Test F1 | Dev F1 | Latency | Cost / 1k |
|---|---|---|---|---|---|---|---|
| **jev-statement-code ★** | Jev P(yes) per tag + thresholds | No (thresholds) | n/d | **0.7515** | 0.7293 | 0.26 s | $0.13 |
| f2llm_late_fusion | dense emb. ridge + TF-IDF, blended | No (frozen) | 596M | 0.7477 | 0.7057 | 0.14 s | free, GPU |
| f2llm_probe | dense embedding + ridge classifier | No (frozen) | 596M | 0.7415 | 0.7087 | 0.13 s | free, GPU |
| descode2tag-json † | gpt-6-luna: statement + code → JSON | No (zero-shot) | n/d | 0.7407 | 0.7319 | 1.21 s | $0.13 |
| jev-statement | Jev P(yes) per tag + thresholds | No (thresholds) | n/d | 0.7367 | 0.7289 | 0.25 s | $0.12 |
| f2llm_early_fusion | dense embedding + TF-IDF + linear SVM | No (frozen) | 596M | 0.7301 | 0.7022 | 0.14 s | free, GPU |
| descode2tag-text | gpt-6-luna: statement + code → text | No (zero-shot) | n/d | 0.7209 | 0.7061 | 0.99 s | $0.11 |
| modernbert_base_full | cross-encoder classifier, 5 folds | Yes (full) | 150M | 0.7039 | 0.6530 | 0.36 s | free, GPU |
| modernbert_lora | LoRA cross-encoder classifier, 5 folds | Yes (LoRA) | 396M | 0.6954 | 0.6611 | 0.33 s | free, GPU |
| tfidf | TF-IDF words + code + linear SVM | From scratch | 366K | 0.6888 | 0.6629 | 0.007 s | free, CPU |
| code2tag-json | gpt-6-luna: code only → JSON | No (zero-shot) | n/d | 0.6668 | 0.6514 | 1.14 s | $0.08 |
| code2tag-text | gpt-6-luna: code only → text | No (zero-shot) | n/d | 0.6314 | 0.6421 | 1.09 s | $0.06 |
| label_prior | constant: predicts all 8 tags | No model | — | 0.1487 | 0.1485 | < 1 ms | free, CPU |

★ CLI default · † best dev F1 · Dev F1: out of fold · n/d: size not disclosed by the provider

# 04

# Methods and code

Six families · score by tag · Jev · code · MLflow · command line · serving

# The six method families

Test F1 climbs from the 0.15 floor to 0.75

### Label prior

`label_prior`

No model: predicts all 8 tags, reads no text.

**0.15 · free**

### TF-IDF

`tfidf`

366K linear weights over word and code-token TF-IDF.

**0.69 · CPU, 7 ms**

### Cross-encoder classifier (ModernBERT)

`modernbert_lora, _base_full`

base 150M or large 396M; a bag of 5 fold models.

**0.70 · GPU**

### Dense embeddings (F2LLM)

`f2llm_probe, _early, _late`

Frozen 596M encoder; ridge probe or fused with TF-IDF.

**0.73–0.75 · GPU, 0.13–0.14 s**

### LLM-based (Code2Tag, DesCode2Tag)

`code2tag-*, descode2tag-*`

Zero-shot gpt-6-luna, xCodeEval prompts; size not disclosed.

**0.63–0.74 · $0.06–0.13 / 1k**

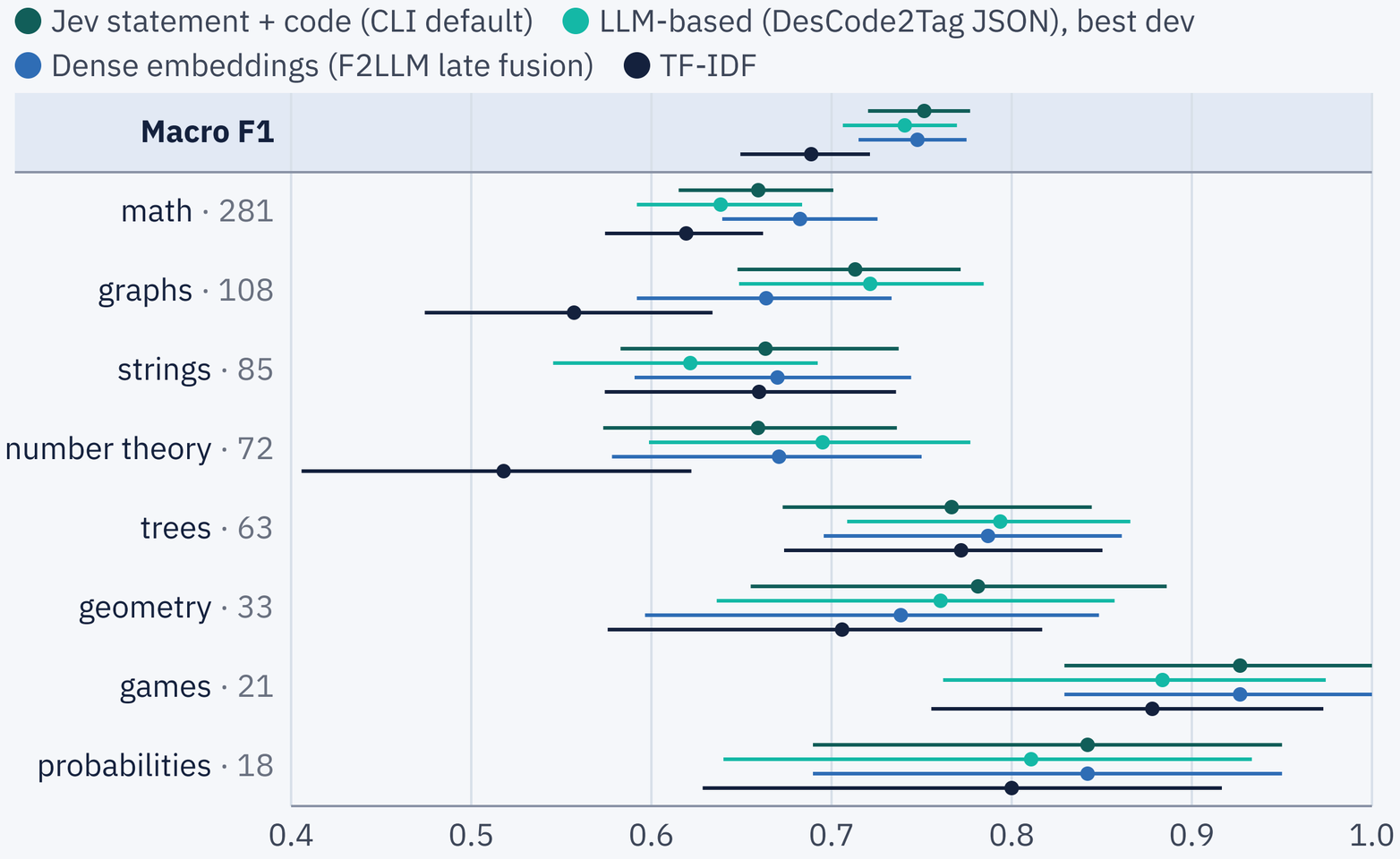
### TypeSafe Jev

`jev-statement, -code`

One yes/no question per tag; size not disclosed.

**0.74–0.75 · $0.12–0.13 / 1k**

# Score by tag




## Where LLMs gain

Jev over TF-IDF: graphs +0.16, number theory +0.14

## No clear winner

strings, trees, geometry: every interval overlaps

## Rare tags, noisy

probabilities: 18 test problems, ±0.13 to 0.15

# How Jev works

## One call, 8 tag questions; only thresholds are learned

```
import typesafe_sdk
# CONVENTION, QUESTION, TAG_DEFINITIONS: typesafe_jev/settings.py
def noul(tag):                              # one yes/no question per focus tag
    return {"type": "noul",
            "instructions": {"tag": tag,
                             "convention": CONVENTION,
                             "question": QUESTION.format(tag=tag)},
            "criteria": TAG_DEFINITIONS[tag]}   # "true" / "false"

questions = {tag.replace(" ", "_"): noul(tag) for tag in FOCUS_TAGS}

async with typesafe_sdk.AsyncTypeSafeClient(model="jev-1.13.0") as client:
    response = await client.system_one(
        state={"statement": statement, "solution": source_code},
        model="jev-1.13.0",
        questions=questions)

p = {tag: response.nouls[tag.replace(" ", "_")].noul   # P(yes)
     for tag in FOCUS_TAGS}
```

### The question, for number theory

*Would Codeforces put the tag `number theory` on this problem?*

**Yes:** The intended solution relies on properties of integers.

**No:** Integers are only quantities to add, compare or count.

### Adding the solution

Dev macro F1: 0.729 both. Test: 0.737 without, 0.751 with.

# Code structure

## One package shared by notebooks, methods and CLI

### 1 · Data

`data/`
`preprocessing/`

Clean text; read Python 2 and 3.

→

### 2 · Split

`near_duplicates/`
`splitting.py`

Freeze dev, test, 5 folds.

→

### 3 · Methods

`methods/ model/`
`llm/`

14 registered, 13 frozen.

→

### 4 · Report

`evaluation/`
`cli.py`

Per-tag F1, bootstrap intervals.

**`tuning/ tracking/`**
Optuna and MLflow

**`assets/`**
The committed record

**`notebooks/`**
01–09, frozen files

**`tests/ docs/adr/`**
110 test files, 8 ADRs

### One contract

Every method predicts, saves, freezes and verifies alike.

### Light prediction path

MLflow, Optuna and SDKs imported only where needed.

### Verifiable

verify re-derives every frozen number offline.

# One interface for all methods

```python
@runtime_checkable
class TaggingMethod(Protocol):              # every method, any kind
    @property
    def name(self) -> str: ...              # its key in METHODS
    @property
    def tags(self) -> tuple[str, ...]: ...  # its output columns
    def predict(self, raw_problems: pd.DataFrame) -> pd.DataFrame: ...
    def save(self, folder: Path) -> None: ...   # data files only

@runtime_checkable
class ScoreMethod(TaggingMethod, Protocol):     # kind "score"
    def scores(self, raw_problems: pd.DataFrame) -> pd.DataFrame: ...

METHODS = {                                 # one line per method
    "tfidf": MethodSpec("codeforces_tags.model.tag_model", "score", (...)),
    "jev-statement-code": MethodSpec(
        "codeforces_tags.methods.typesafe_jev.method", "score", (...),
        groups=("llm-typesafe",), calls_api=True),
}

save_method(method, folder)   # method.json, then method.save(folder)
method = load_method(folder)  # the registry's module.load(folder)
check_tagging_method(method, raw_problems, tmp_path / "model")
```

### Raw in, 0/1 tags out

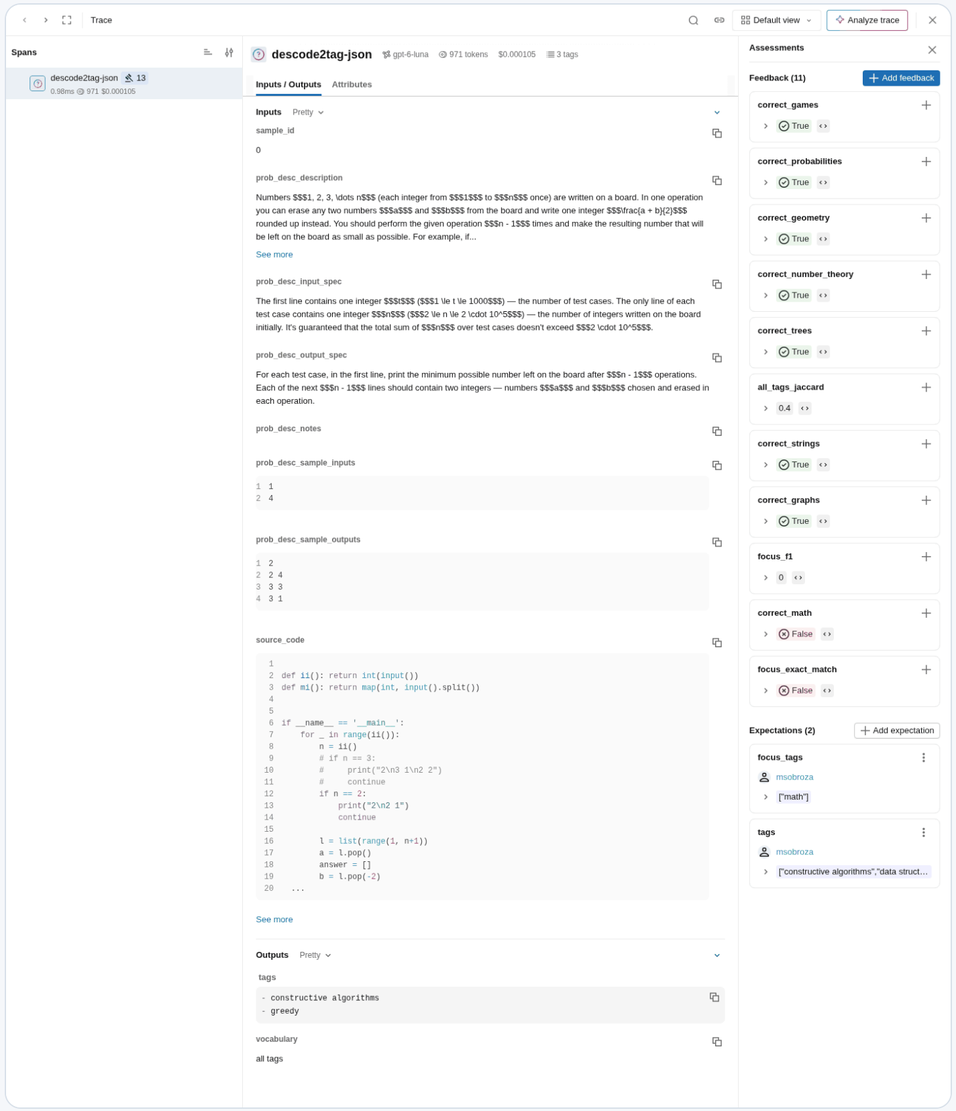
TF-IDF, fine-tuned bags, API models: same predict and save.

### A registry, not paths

load_method imports the module the registry names.

### An executable contract

Every method's tests call check_tagging_method.




MLFLOW

# Predictions in MLflow

## descode2tag-json misses math on dev problem 0

### Graded per tag

One filter, correct_math = false: 798 dev problems.

### Cost and tokens per call

Here gpt-6-luna, 971 tokens, $0.000105.

### Rebuilt for free

From committed files: an empty store needs no paid call.

# Command line

## One entry point, codeforces-tags, for every method

### List

```
codeforces-tags methods
```

Every method, its dev and test F1, its needs.

### Predict

```
codeforces-tags predict sample_42.json
codeforces-tags predict problems/ --method tfidf
```

Default jev-statement-code; tfidf runs offline.

### Evaluate

```
codeforces-tags evaluate --method tfidf --out report/
codeforces-tags evaluate my_labelled_problems/
```

Per-tag precision, recall, F1. Test part by default.

### Train and freeze

```
python -m codeforces_tags.tuning run --trials 200
python -m codeforces_tags.tuning freeze
python -m codeforces_tags.methods.<package> freeze
python -m codeforces_tags.methods.<package> verify
```

Tune on dev, freeze once; verify re-derives it.

Prefix each command with `uv run`; an API method also needs its group, its key and `--max-cost-usd`.

# Serving a method

## One frozen method, one Docker image, three targets

### BUILT FROM THE COMMITTED FILES

**Frozen method**
the folder in assets/methods

→

**MLflow model**
pinned requirements, hashed weights

→

**Docker image**
MLflow's server behind nginx, CPU or GPU

→

**POST /invocations**
tags, scores and thresholds

### RUNS ON

**Docker Compose**
on localhost only; a CPU and a GPU profile

**Kubernetes**
kustomize, CPU and GPU overlays, ClusterIP

**SageMaker**
aws CLI scripts, behind IAM

**Clients**
a Streamlit app · smoke: 20 test problems against the frozen predictions · links to the run and prompts in MLflow

**Run so far**
tfidf (906 MB image): Docker and a kind cluster, CPU
f2llm_late_fusion (7.89 GB): Docker, CPU and GPU
median 3.40 s on the CPU, 0.148 s on the GPU

# Serving in practice

## One command to try a method, four to check it step by step

### Try it model server, MLflow UI and Streamlit app in one terminal, all stopped on Ctrl-C

```
uv run --group app python -m codeforces_tags.serving up --method f2llm_late_fusion
```

Checks everything first and says what is missing · `--build` builds the image · `--local` without Docker · `--device gpu` on the GPU

### Step by step, with tfidf

**Build**

```
uv run python -m codeforces_tags.serving build --method tfidf
```

**Run**

```
METHOD=tfidf docker compose -f deploy/docker-compose.yml --profile cpu up --wait
```

**Check**

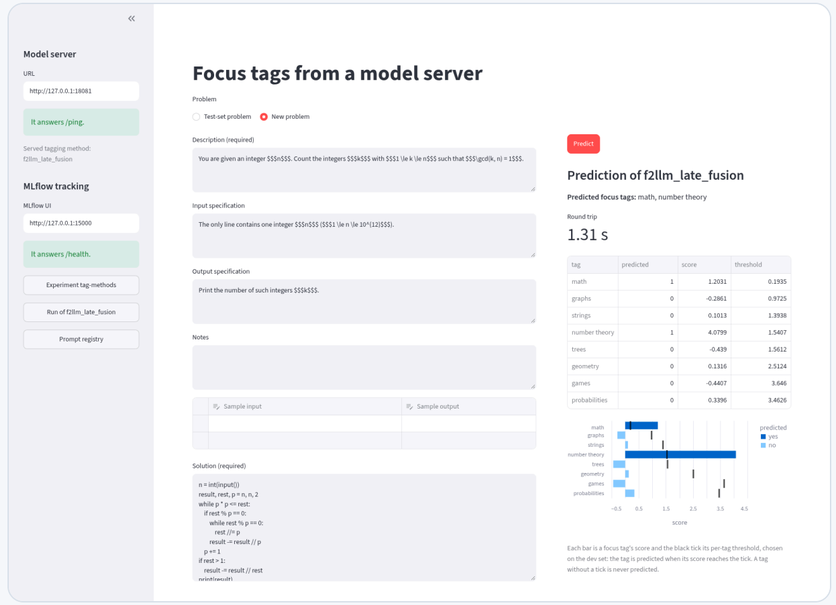
```
uv run python -m codeforces_tags.serving smoke --method tfidf
```

**Ask**

```
uv run --group app streamlit run apps/streamlit_app.py
```

POST /invocations answers, for a tree problem with tfidf:

```
{"math": 1, "trees": 1, "score: trees": 0.333, "threshold: trees": -0.357, ...}
```




The Streamlit app asking f2llm_late_fusion, served **on the CPU**: a typed problem, 1.31 s round trip

# Conclusion

## What we learned

### The statement carries the signal

Code alone 0.63–0.67; with the statement 0.72–0.74.

### Thresholds matter as much as models

Same model: 0.19 at one 0.5 cutoff, 0.61 per tag.

### A local GPU nearly matches Jev

Dense embeddings: 0.748 vs 0.751, 2× faster, free.

### Full fine-tuning was not needed

Frozen dense embeddings (0.73–0.75) beat the fine-tuned cross-encoder (0.70).

### LLMs gain on two tags

graphs and number theory: +0.11 to +0.18 over TF-IDF.

### Prompt search did not help

GEPA on Jev's definitions: +0.006, within the noise.

**0.75** best test macro F1 (Jev)

**0.26 s** CLI default, $0.13 per 1k

**Dev decides** test scored once